LLM Day NYC 2026 Q1

Multi-Agent Architectures: Solving Production LLM Reliability at Scale

Moving beyond single-prompt fragility to robust, self-correcting systems using decomposition, routing, and peer review.

Pranav Kowadkar
AI Engineer · NJIT

The Production Reliability Crisis

Scale without orchestration is just chaos

41–87%

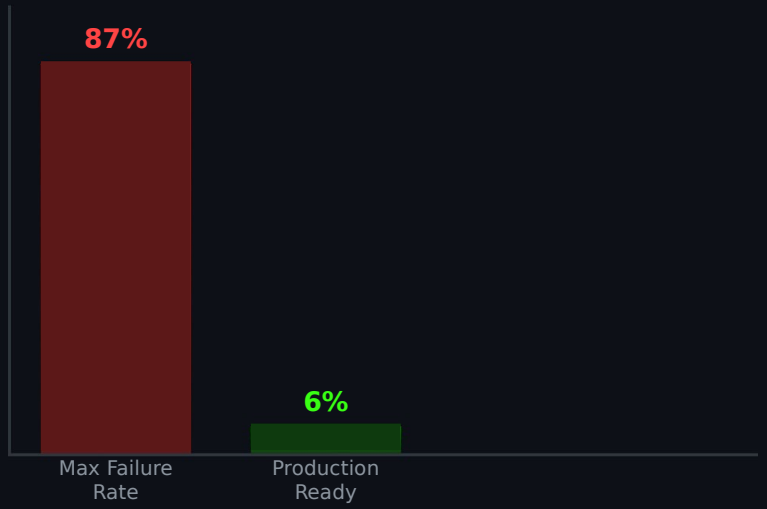
Production Failure Rate

Context overflow & cascading failures

No output verification mechanism

Sequential processing bottlenecks

57% have agents, only 6% are ready



Source: MAST / UC Berkeley & CMU (arXiv:2503.13657) · PagerDuty / InfoQ AI Trends 2025

"This is an architecture problem, not a model problem."

What Is an AI Agent?

Goal-directedness + Tool Use + State

1. Goal-Directed

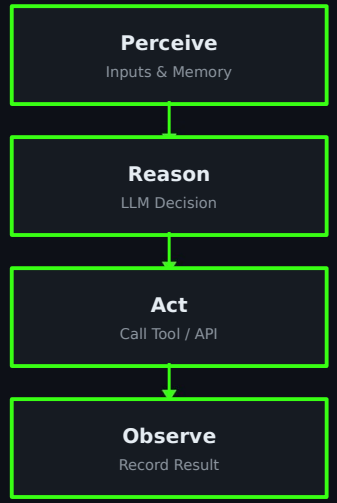
Works toward an objective across multiple steps without human intervention at each step.

2. Tool Use

Calls external functions, APIs, and databases — incorporating results into its reasoning.

3. State

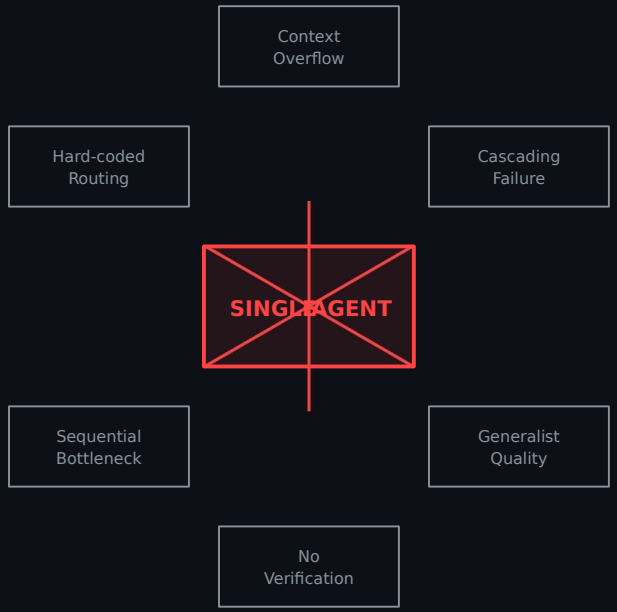
Maintains memory of history, tool outputs, and partial results across the full task lifecycle.



> *"The quality of an agent's reasoning is bounded by the quality of the information it perceives. Garbage in, garbage out."*

Why Single Agents Fail at Scale

Single Point of Failure = Total System Failure



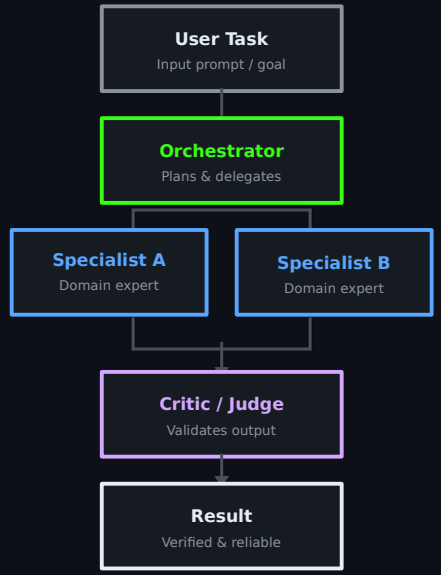
Single-Agent Problem	What Actually Happens
Context Overflow	Long tasks exceed window; reasoning degrades
Cascading Failure	One error kills the entire task; no recovery
Generalist Perf.	Jack of all trades, master of none
No Verification	Bad output reaches user unchecked
Seq. Bottleneck	Latency compounds; user waits
Hard-coded Routing	New task types break existing logic

A2A Orchestration: The Structural Fix

Reliability through structure, not capability.

❌ Single Agent	✅ A2A Architecture
One failure = total failure	Isolated retry at failing stage
Generalist — jack of all trades	Domain specialist per task
No output verification	Critic / Observer validation
Sequential bottleneck	Parallel execution
Hard-coded routing	Confidence-score routing

> *"A2A changes the shape of the system so failures are isolated and outputs are verified."*



The 12 Patterns: Three Tiers

Organized by production maturity, not complexity



> *"Match pattern complexity to your demonstrated reliability gap. Start with Sequential. Add complexity only when you can measure the failure it solves."*

Pattern #1: Sequential Handoffs

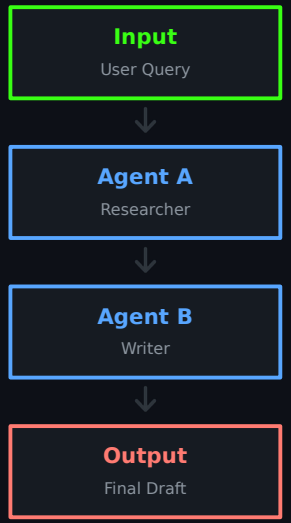
The "Chain of Responsibility" for LLMs.

> **Linear Flow:** Output of Agent A becomes Input of Agent B. No loops, no conditions.

> **Ideal For:** Well-defined pipelines like "Research -> Draft -> Polish".

> **Failure Mode:** Error propagation. If Agent A hallucinates, Agent B amplifies it.

```
chain = (
 PromptTemplate("Research {topic}")
 | model
 | PromptTemplate("Summarize {text}")
 | model
 )
```

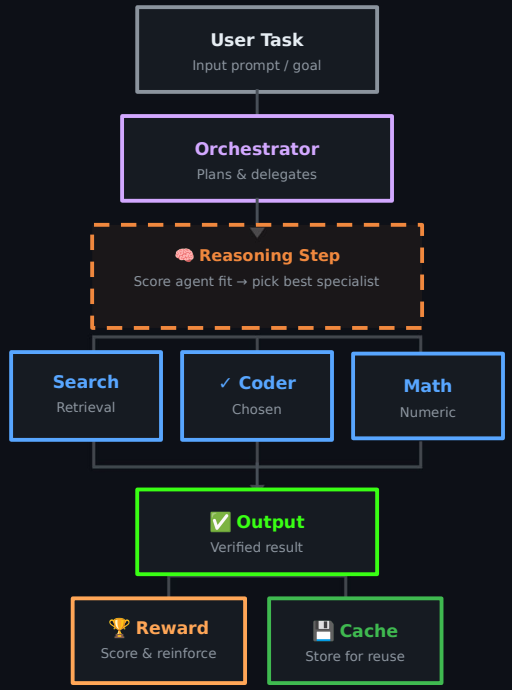


Pattern #2: Hierarchical

The "Manager-Worker" Architecture.

- **Central Brain:** An Orchestrator breaks down the task, reasons about which specialist to call, and delegates.
- **Reasoning Step:** Before dispatching, the Orchestrator scores each agent's fit — not just hard-coded routing.
- **Output Validation:** The chosen agent's output is verified before being returned to the user.
- **Reward + Cache:** Good outputs are scored and cached — the system learns which agent wins for which task type.

> *"Don't ask the CEO to write the code. Ask the CEO to hire the right Coder."*



Pattern #3: Collaborative

Two or more agents, one goal — iterate until it's right.

> **Two Agents, One Task:** Agent A generates a response. Agent B reviews it and provides structured feedback.

> **Regenerate with Feedback:** Agent A receives the critique and regenerates — a targeted revision, not a blind rewrite.

> **Decision Gate:** After each round — "good enough" exits the loop; "needs work" sends it back for revision.

> **Loop Limit:** Always cap iterations (e.g., 3 rounds) to prevent infinite cycles in production.

✓ WHEN TO USE

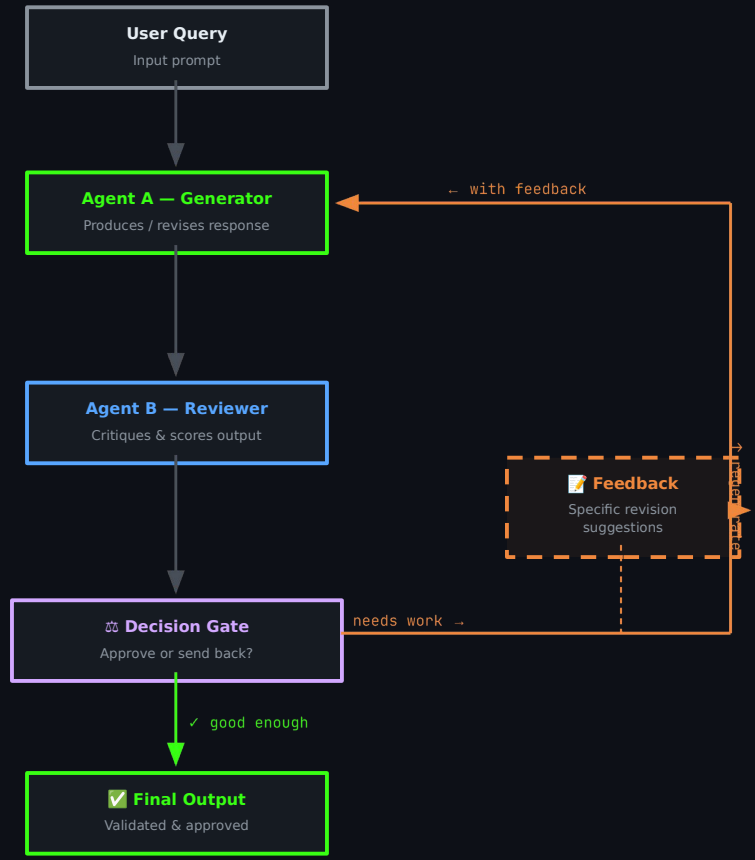
Quality matters more than speed

Output needs peer validation

✗ WHEN NOT TO USE

Latency-critical pipelines

Simple factual lookups



Pattern #4: Competitive

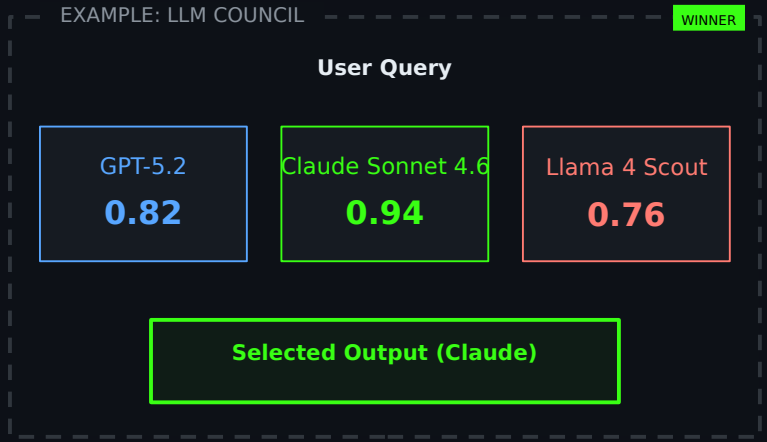
May the best agent win.

> **Parallel Execution:** Multiple agents attempt the same task independently.

> **Confidence Scoring:** Each agent (or a judge) scores the output.

> **Selection:** The system picks the highest-confidence answer.

> *"Redundancy is the price of reliability."*



Pattern #5: Event-Driven / Reactive

Don't wait for a prompt. Act on the signal.

> **Agents subscribe** to data streams — logs, webhooks, Slack — and fire automatically when a matching event arrives.

> **Fully decoupled:** The event source has no knowledge of which agents are listening. New agents subscribe without touching the source.

> **Background intelligence:** Agents proactively surface insights — monitoring, alerting, auto-triage — without being prompted.

> **Fan-out by design:** One event triggers multiple agents simultaneously, each handling a different concern.

✓ WHEN TO USE

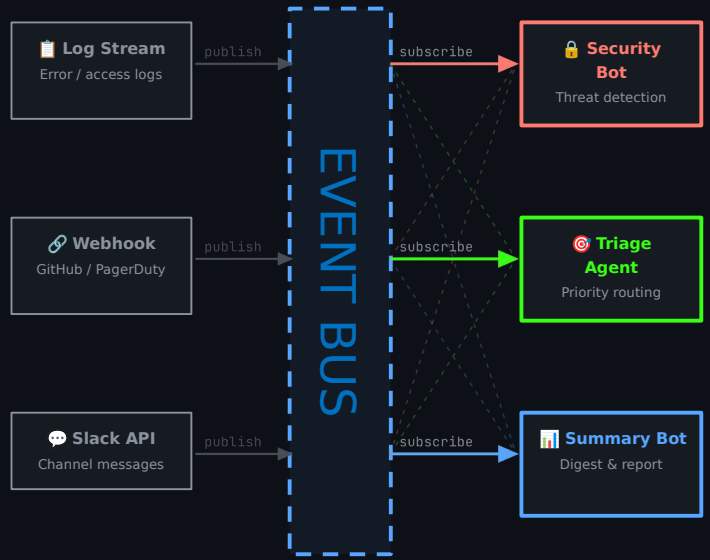
System monitoring & alerting

Customer support auto-triage

✗ WHEN NOT TO USE

Linear, step-by-step workflows

Strict ordering requirements



Pattern #6: Fan-out / Fan-in

Map-Reduce for the AI age.

> **Fan-out:** Split one complex task into N independent sub-tasks — e.g., "Research 5 different companies in parallel."

> **Fan-in:** Collect all parallel outputs and synthesize them into a single coherent final answer.

> **Speed wins:** Total latency = slowest single worker, not the sum of all. 5 tasks in parallel ≈ 1 task's time.

> **Aggregator matters:** The fan-in step needs a strong synthesizer — poor aggregation wastes all the parallel work.

✓ WHEN TO USE

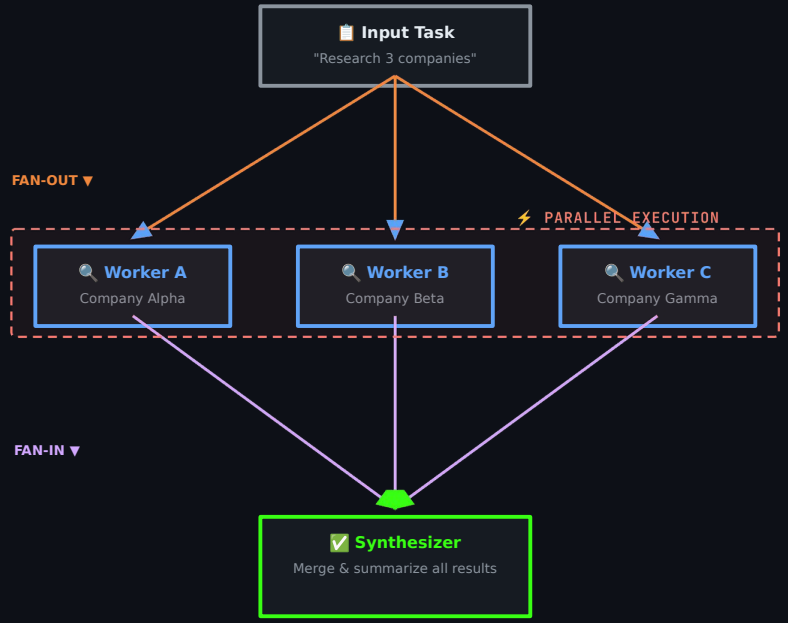
Independent sub-tasks (research, analysis)

Batch processing at scale

✗ WHEN NOT TO USE

Tasks with sequential dependencies

When sub-tasks share mutable state



Pattern #7: Hybrid Routing

Local for speed & privacy. Cloud for intelligence.

> **Route simple tasks** — extraction, PII scrubbing, classification — to small local models (Llama 3 8B, Phi-3).

> **Route complex reasoning** — multi-step analysis, creative generation — to frontier cloud models (Claude, Gemini).

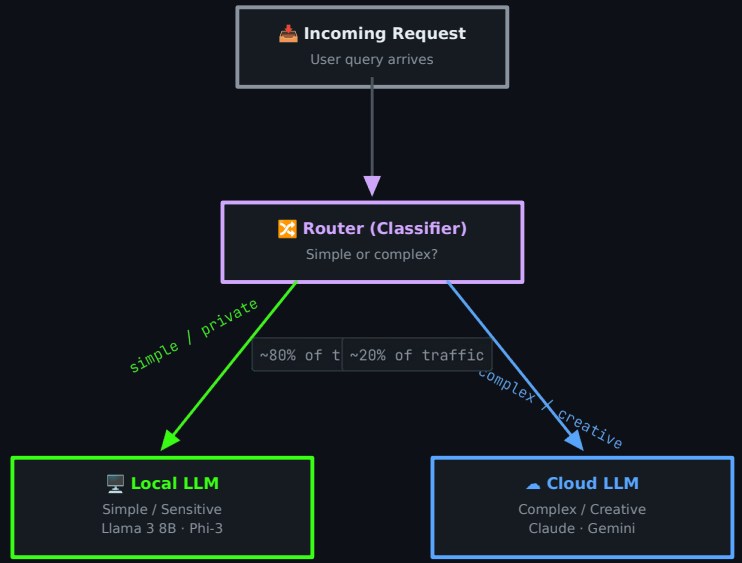
> **Router logic:** A lightweight classifier or perplexity threshold decides which path each request takes.

> **Privacy bonus:** Sensitive data never leaves your infrastructure — only non-sensitive requests go to the cloud.

IMPACT

~10x Cost Reduction

for high-volume, mixed-complexity workloads



Pattern #8: RLVR-Style Feedback

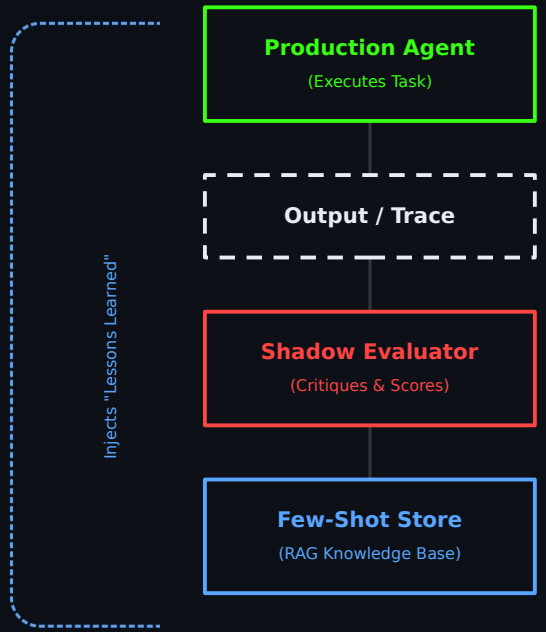
Continuous improvement without fine-tuning.

> **Shadow Observer:** A separate agent critiques interactions asynchronously (post-hoc).

> **Dynamic Few-Shot:** High-quality critiques and corrections are saved to a vector store.

> **In-Context Learning:** Next time a similar query comes in, the agent retrieves past "lessons learned".

> *"The system gets smarter the more it runs, turning production logs into training data automatically."*



Pattern #9: Adversarial Debate

Truth emerges from conflict.

> Assign opposing personas ("Pro" vs "Con") to argue a specific point.

> **Why?** Single agents tend to be sycophantic (agreeing with the user). Debate forces critical analysis.

> A neutral **Judge Agent** synthesizes the arguments into a final verdict.



> *"The best way to find a hallucination is to pay another agent to find it."*

Pattern #10: Actor-Critic

Write. Critique. Revise. Repeat.

> **Actor** generates the initial draft or solution — fast, creative, unconstrained.

> **Critic** reviews the output against specific criteria — "Is it secure?", "Is it concise?", "Is it correct?"

> **Feedback loop:** Critique is fed back to the Actor for targeted revision — not a blind rewrite.

> **Pushback allowed:** The Actor can defend its choices. The Critic can respond. Dialogue, not dictatorship.

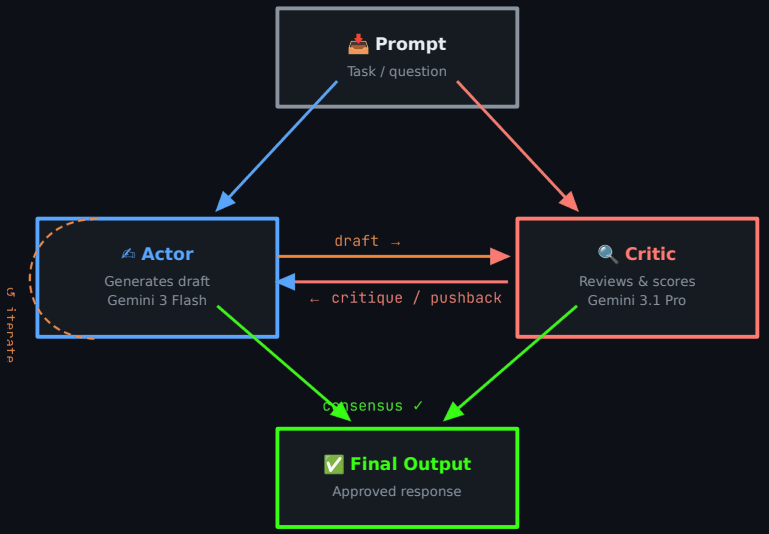
⚡ LIVE DEMO -- TRY IT NOW

Actor vs. Critic debate -- live on stage & online.

⚡ Actor: Gemini 3 Flash

🔍 Critic: Gemini 3.1 Pro

actor-critic.manus.space



"Write drunk, edit sober." — Hemingway (the original Actor-Critic)

Pattern #11: Blackboard

Implicit coordination via shared state.

> **Indirect communication:** Agents never talk to each other directly — they only read from and write to a central Blackboard.

> **Loose coupling:** Add a new specialist agent without rewiring anything. It just subscribes to the shared state.

> **Control loop:** A monitor watches the blackboard and activates the right agent when new data appears.

> **Scales cleanly:** 10 agents = 10 connections to the board, not 45 peer-to-peer channels.

✓ WHEN TO USE

Many specialists, evolving shared context

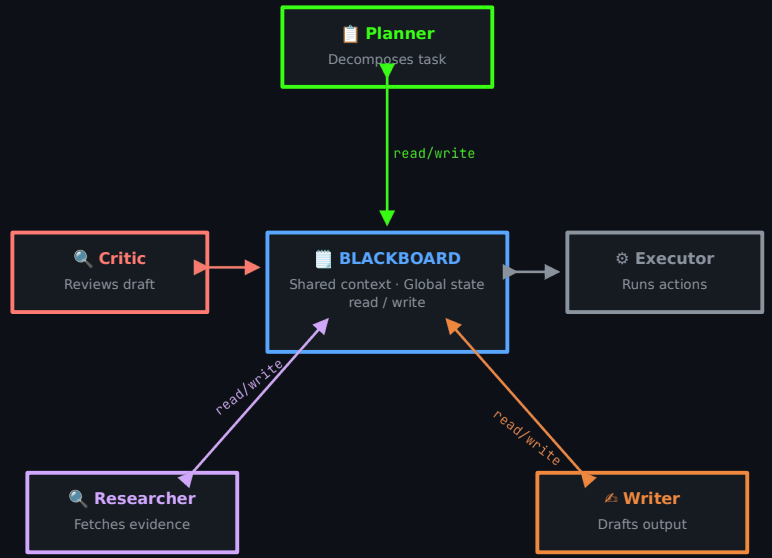
Complex problem-solving (planning, diagnosis)

✗ WHEN NOT TO USE

Simple linear pipelines

When shared state creates race conditions

"If you have 10 agents, you don't want 45 channels. You want 1 shared state."



5 agents · 5 connections (not 10)

Pattern #12: Swarm

Emergent intelligence. Zero orchestration.

> **No central brain:** There is no Orchestrator or Router. Every agent is equal and self-directed.

> **Local rules only:** Each agent follows simple peer logic — "If a neighbor finds X, verify it. If stuck, ask a neighbor."

> **Emergent behavior:** Complex, intelligent outcomes arise from many simple interactions — no one planned the result.

> **Resilient by design:** Kill 50% of agents — the system slows down but never stops. No single point of failure.

✓ WHEN TO USE

Massive parallel web scraping

Market / social simulations

Brainstorming via cross-pollination

✗ WHEN NOT TO USE

Tasks requiring strict ordering

When you need a deterministic result



How to Choose: The Decision Matrix

Map your failure mode → pick the right pattern.

#	IF YOU SEE THIS FAILURE MODE…	USE THIS PATTERN…	BECAUSE…	PRIMARY CONCERN
01	**Hallucination & logic errors**	**Adversarial Debate or Actor-Critic**	Conflict forces the model to justify its reasoning and catch its own errors before output is accepted.	quality
02	**Context window overflow**	**Fan-out / Fan-in (Map-Reduce)**	Splits large inputs into parallel chunks — each agent handles a slice, a synthesizer merges results.	scale
03	**High latency & cost**	**Hybrid Routing**	Routes simple tasks to small local models, reserving expensive frontier calls for genuinely hard problems.	cost
04	**"Lost in the weeds" / infinite loops**	**Hierarchical (Orchestrator)**	A manager agent holds the high-level plan and intervenes when sub-agents drift or loop.	complexity
05	**Unpredictable / edge-case inputs**	**Competitive (Bidding)**	Multiple agents bid on the task — the most confident specialist wins, handling the unexpected gracefully.	resilience
06	**Reactive / event-triggered needs**	**Event-Driven (Reactive)**	Agents subscribe to data streams and fire automatically — no polling, no manual orchestration.	scale

Patterns are not mutually exclusive — production systems typically combine 2–3 patterns. Start simple; add complexity only when a specific failure mode demands it.

⚠ The Anti-Patterns

How to build a very expensive Rube Goldberg machine.

ANTI-PATTERN 01

The God Orchestrator

The router becomes so smart it starts doing the work itself — 500 lines of if/else instead of delegating.

routers route, specialists specialize

ANTI-PATTERN 02

Infinite Loops

Agent A calls Agent B, who calls Agent A. No exit condition. Your bill grows forever.

always set a max_turns counter

ANTI-PATTERN 03

Resume-Driven Development

Using 5 LLM agents to parse a JSON file. A regex or a 3-line script would be 100× faster and cheaper.

use the simplest tool that works

ANTI-PATTERN 04

Context Pollution

Passing the entire 50k-token conversation history to every specialist. They only need their slice.

pass minimal, scoped context only

god_orchestrator.py — ✗ BAD

```
def route(user_input):
    # router doing the actual work 🫠
    if "code" in user_input:
        return llm.generate_code(user_input)
    elif "write" in user_input:
        return llm.write_essay(user_input)
    # ... 500 more lines ...
```

proper_router.py — ✓ GOOD

```
def route(user_input):
    # router only classifies, never executes
    intent = classifier.predict(user_input)
    agent  = registry.get(intent)
    return agent.run(
        input=user_input,
        max_turns=10  # ← always cap loops
    )
```

"If your router has 500 lines of logic, it's not a router. It's a monolith with extra steps."

Key Takeaways

TL;DR — what to remember when you leave this room.

TAKEAWAY 01



Architecture > Prompting

Stop trying to prompt-engineer your way out of a reasoning deficit. A well-structured multi-agent system will outperform a single mega-prompt every time — because structure forces decomposition.

→ design the system, not just the prompt

TAKEAWAY 02



Start Sequential, Evolve Deliberately

Don't build a swarm on day one. Build a chain. Add a router only when you can measure the gap. Add a loop only when you can define the exit condition. Complexity is a tax — pay it only when necessary.

→ earn each layer of complexity

TAKEAWAY 03



Log Everything — It's Your Flywheel

Every multi-agent trace you capture today is a fine-tuning dataset tomorrow. The teams winning at AI aren't just building better prompts — they're building data loops that make their models smarter over time.

→ traces today = training data tomorrow

```
ubuntu@llm-day:~$ echo "Now go build something that actually works."
```

Q&A + Resources

Thank you for attending LLM Day NYC 2026.

SPEAKER

Pranav Kowadkar

pkowadkar.com

AI Engineer · Multi-Agent Systems
LLM Day NYC 2026

FIND ME ONLINE

GitHub · Code & Projects
github.com/p-kowadkar

LinkedIn · Connect
inkedin.com/in/pkowadkar

📎 **Post-workshop:** Slides & Actor-Critic source code will be posted on **LinkedIn** and **pkowadkar.com** — downloadable as PPTX.

ubuntu@llm-day:~$ Questions? The floor is yours.

FURTHER READING

Anthropic — Building Effective Agents
Practical patterns for production multi-agent systems from the Claude team.
anthropic.com/research/building-effective-agents

OpenAI — Swarm (Experimental)
Lightweight framework for orchestrating multi-agent handoffs and routines.
github.com/openai/swarm

LangGraph — Stateful Agent Workflows
Graph-based framework for building controllable, stateful agent pipelines.
langchain-ai.github.io/langgraph

Google DeepMind — RLVR & Process Reward Models
Research on reinforcement learning from verifiable rewards for LLM reasoning.
deepmind.google/research

Chip Huyen — AI Engineering (Book)
Comprehensive guide to building production AI systems, agents, and evals.
oreilly.com/library/view/ai-engineering

Google & Linux Foundation — Agent2Agent (A2A) Protocol
Open standard enabling secure communication and interoperability between opaque agentic applications across different frameworks.
github.com/a2aproject/A2A